Lab Session: Knowledge reasoning with rule and Knowledge Graph Embedding (KGE)

(Using the Knowledge Base Constructed in the Previous Lab)

Objective

In this lab, you will firstly apply SWRL to design reasoning rules on a prepared ontology.

Then, you will apply **Knowledge Graph Embedding (KGE)** techniques to the knowledge base constructed and expanded in the previous session.

You will:

- Prepare the KB for embedding
- Train embedding models
- Evaluate link prediction performance
- Analyze the impact of KB structure and size
- Compare different embedding approaches

The quality of your embeddings will strongly depend on the design decisions made during the previous lab.

You may need 3H to 6H to do this lab.

Part 1 - Knowledge reasoning with SWRL

Load file `family.owl` and apply a SWRL rule infering "a person who is older than 60 years old is an 'oldPerson'" ('oldPerson' is a class).

Install `OWLReady2` as the reasoning engine in Python.

Mark your SWRL rule and the reasoning result.

You will also need to use this in Exercise 8 of Part 2.

Part 2 - Knowledge Graph Embedding

1 — Data Preparation

1.1 Input

Use the expanded KB generated in the previous lab:

(The reason of the volume: If the entity number is too small, the embedding make no sense. If the number is too big, it would be hard to train on your personal computers.)

- 50,000 – 200,000 triplets
- 5,000 – 30,000 entities
- 50 – 200 relations

1.2 Cleaning for Embedding

Before training:

- Remove duplicate triplets
- Remove inconsistent URIs
- Ensure all entities and relations are uniquely indexed
- Optionally remove literal-heavy predicates

1.3 Train / Validation / Test Split

Split the dataset:

- 80% Training
- 10% Validation
- 10% Test

Ensure:

- No entity appears only in validation/test
- No leakage between splits

Deliverable:

- `train.txt`
- `valid.txt`
- `test.txt`

---

2 — Embedding Models

You must implement or use at least **two embedding models**.

Recommended options:

- TransE
- DistMult
- ComplEx
- RotatE

You may use frameworks such as:

- PyKEEN (recommended)
- or other frameworks: DGL-KE, OpenKE or Custom PyTorch implementation

---

3 — Training Configuration

Specify:

- Embedding dimension (e.g., 100 or 200)
- Learning rate
- Batch size
- Number of epochs
- Negative sampling strategy

Keep configuration consistent across models for fair comparison.

---

4 — Evaluation Task: Link Prediction

Evaluate models using:

- Mean Reciprocal Rank (MRR)
- Hits@1
- Hits@3
- Hits@10

Perform both:

- Head prediction
- Tail prediction

Report filtered metrics.

---

5 — Experiments to Conduct

You must perform the following analyses:

5.1 Model Comparison

Compare performance of at least two models.

Discuss:

- Which model performs better?

5.2 KB Size Sensitivity

Subsample your KB:

- 20k triples
- 50k triples

- Full dataset

Observe:

- How performance scales

---

6 — Embedding Analysis

With your **best trained embedding model**, do the following analysis:

6.1 Nearest Neighbors

For selected entities:

- Retrieve nearest neighbors in embedding space
- Analyze semantic coherence

Example:

Nearest neighbors of MarieCurie:

1. AlbertEinstein
2. RosalindFranklin
3. NielsBohr

Discuss whether results are meaningful.

6.2 Clustering analysis

After training your KGE model:

1. Extract entity embeddings
2. Apply t-SNE to reduce dimensionality to 2D
3. Plot entities on a 2D scatter plot

Color the entities according to their ontology class, and answer the following question:

- Do entities of the same class cluster together?
- Does the embedding capture semantic structure?

(You may encountered unexplainable results. You should also mention this even though no explicit result is demonstrated.)

6.3 Relation Behavior

Analyze relation types:

- Symmetric relations
- Inverse relations
- Composition relation of multiple relations Discuss which models handle them better.

---

7 — Critical Reflection

Discuss:

- Impact of predicate alignment quality
- Impact of noisy expansion
- Effect of ontology modeling choices
- Open-world assumption vs embedding assumptions

Explain how the previous lab design influenced embedding performance.

---

8 - Comparison between rule-based and embedding based reasoning

Design a SWRL rule for your knowledge base (don't make it too complicated, a horn rule with two conditions is sufficient) Can you find a similar result with your embedding result? Example: for rule `Person(?p) ∧ hasSibling(?p,?s) ∧ Man(?s) → hasBrother(?p,?s)` Can you find a result giving `vector(hasSibling)+vector(Man)`$\approx$ `vector(hasBrother)`?

Deliverables

1. Cleaned triple dataset with Train/validation/test splits
2. 4–6 page report including (To be integrated in your final project report):
    - Evaluation results table
    - Methodology
    - Hyperparameters
    - Experimental results
    - Analysis
    - Conclusions

Expected Observations

- Small KBs produce **unstable** embeddings
- TransE struggles with complex relations
- DistMult fails on antisymmetric relations
- ComplEx handles asymmetric relations better
- Larger KB improves stability but increases training time

Learning Outcomes

After this lab, you should understand:

- How KGE models learn structure
- How KB quality impacts embeddings
- Differences between translational and bilinear models
- Practical limitations of KGE on real data

Important

Your results will be highly dependent on:

- KB size
- Predicate alignment correctness
- Graph connectivity
- Noise introduced during expansion

A poorly constructed KB will lead to poor embeddings.